SismoLab AVL

Manual técnico

Edición: octubre de 2026

Alcance: implementación actual del proyecto Python

Este manual describe cómo ejecutar, entender y verificar SismoLab AVL. La aplicación es una herramienta local de demostración y pruebas; no sustituye sistemas de monitoreo sísmico de producción.

Contenido

1. Requisitos y ejecución
2. Organización del proyecto
3. Arquitectura y flujo de una operación
4. Modelo de eventos y prioridad
5. Árbol AVL, BST y modo estrés
6. Reportes y cola FIFO
7. Asociaciones, consultas y archivo
8. Interfaz
9. Persistencia e importación
10. Pruebas y archivos de ejemplo
11. Referencia rápida de módulos

1. Requisitos y ejecución

- Python 3.12 o superior.
- Tkinter disponible en la instalación de Python.
- No se requieren paquetes externos para iniciar la aplicación.

Desde la raíz del repositorio, abra una terminal y ejecute:

```
python main.py
```

El punto de entrada main.py carga la interfaz definida en ui.app. También se puede iniciar con:

```
python -m ui.app
```

2. Organización del proyecto

Directorio	Responsabilidad
core/	Estructuras de datos: árboles, nodos, claves, pila y cola enlazada.
domain/	Reglas de negocio, eventos, prioridades, asociaciones, consultas, métricas y archivo.
services/	Escenario transaccional, historial de deshacer, snapshots y persistencia JSON.
ui/	Interfaz Tkinter, controlador, distribución y dibujo.
data/	Escenarios y ráfagas JSON de ejemplo.
scripts/	Generación de datos de prueba y casos reproducibles.
tests/	Pruebas unitarias y pruebas de integración de la interfaz simulada.

La dirección general de dependencias es:

```
ui -> services -> domain -> core
```

La interfaz no implementa las reglas del dominio: llama al controlador; el controlador convierte errores esperados en resultados legibles para la interfaz.

3. Arquitectura y flujo de una operación

12. La persona usa una pantalla en ui/app.py.
13. La pantalla invoca una operación de ui/controller.py.

14. El controlador delega al escenario de services/scenario.py o al servicio de persistencia.
15. El dominio valida y aplica la regla solicitada sobre el catálogo y el árbol.
16. El escenario registra una instantánea anterior para permitir deshacer.
17. El controlador devuelve un resultado con estado, mensaje y, cuando aplica, datos del resultado.
18. La interfaz actualiza tablas, indicadores y detalles.

Las mutaciones principales del escenario son atómicas: si una acción falla, se restaura el snapshot previo y no se agrega una acción válida al historial.

4. Modelo de eventos y prioridad

Un evento incluye identificador, magnitud, profundidad, coordenadas, hora UTC, revisión, estaciones, estado de atención, prioridad calculada y referencia opcional.

Campo	Regla
Identificador	Entero de 1 a 999999; también acepta texto numérico o prefijo SIS-.
Magnitud	Número finito de -2.0 a 10.0, con máximo un decimal.
Profundidad	De 0.0 a 700.0 km, con máximo un decimal.
Coordenadas X/Y	De 0.0 a 1000.0 km, con máximo un decimal.
Hora	UTC en formato AAAA-MM-DDTHH:MM:SSZ; no puede superar el reloj simulado.
Revisión	Entero positivo.
Atención	pendiente o revisado.

Las zonas son rectángulos dentro del plano 0–1000 km. Los bordes pertenecen a la zona. Un punto se considera poblado si pertenece al menos a una zona marcada como poblada.

Prioridad:

Prioridad	Condición
3, alta	Magnitud ≥ 6.0; o magnitud ≥ 4.5, profundidad ≤ 30 km y epicentro en zona poblada.
2, media	Magnitud ≥ 4.5 y no cumple la condición de prioridad alta.
1, baja	Magnitud < 4.5.

La prioridad y la magnitud forman parte de la clave del árbol; al cambiar esos datos, el evento puede cambiar de posición.

5. Árbol AVL, BST y modo estrés

La clave ordenada es K = (P, M, I): prioridad, magnitud e identificador. El orden lexicográfico ascendente determina el recorrido inorden y distingue eventos con igual prioridad y magnitud.

- En modo normal, el AVL actualiza alturas y rebalancea al insertar o eliminar.
- El factor de balance es altura izquierda menos altura derecha. Una hoja tiene altura 0 y un árbol vacío, -1.
- Se registran los casos LL, RR, LR y RL, además del número de rotaciones elementales.
- El modo estrés aplaza las rotaciones para permitir observar un árbol desbalanceado.
- Recuperar AVL rebalancea el árbol, ejecuta una auditoría y deja la cola pausada. Reanudar cola permite volver al procesamiento continuo.
- La pestaña BST compara el AVL con un árbol binario de búsqueda sin balanceo.

Las operaciones de recorrido e inserción/eliminación del árbol están implementadas de forma iterativa para manejar árboles altos sin depender de la recursión de Python.

6. Reportes y cola FIFO

La cola enlazada conserva el orden de recepción: el primer reporte agregado es el primero que se procesa.

Cada reporte contiene estación, identificador de evento, revisión, magnitud, profundidad, X, Y y hora. En la pestaña Cola:

19. Introduzca un reporte en Preparar ráfaga y pulse Agregar al borrador. Se pueden añadir varios.
20. Pulse Enviar borrador a la cola para validarlos estructuralmente y encolarlos en una sola acción deshacible.
21. Use Paso (1 reporte) para procesar el primero o Continuo para procesar reportes sucesivos.
22. Pausa detiene el ciclo y cancela el siguiente tick programado.
23. La tabla muestra la cola pendiente; la tabla de resultados registra los reportes procesados.

El modo continuo procesa un reporte por tick, evita iniciar ciclos duplicados y se detiene cuando se vacía la cola. Después de una recuperación AVL, el modo continuo permanece bloqueado hasta reanudarlo.

Decisiones posibles al procesar un reporte:

Situación	Resultado
Identificador nuevo y datos válidos	alta: crea un evento.
Misma revisión y mismos datos físicos	confirmado: agrega estación si corresponde.
Misma revisión con datos físicos distintos	conflicto: rechaza el reporte.
Revisión menor a la vigente	antiguo: descarta el reporte sin cambiar el evento.
Revisión mayor	corregido o reactivado si el evento estaba archivado.
Identificador eliminado	rechazado.
Estación, revisión o datos inválidos	invalido.

Cada paso procesa exactamente un reporte y se puede deshacer; el snapshot restaura tanto el catálogo como la posición de los reportes en cola.

7. Asociaciones, consultas y archivo

Una referencia candidata debe ser otro evento de mayor magnitud, ocurrido antes y dentro de la ventana temporal W; la distancia euclidiana entre epicentros no debe superar R. La referencia elegida es la de mayor magnitud; en empate, la más reciente y luego el identificador menor. Cambiar W o R recalcula las asociaciones.

L es el umbral de profundidad en el AVL para el indicador de acceso costoso: se señalan eventos de prioridad 3 situados a una profundidad mayor que L.

Las consultas disponibles incluyen eventos pendientes de mayor clave, intervalo de magnitud, rango de fechas/profundidad, asociaciones y accesos costosos. La pestaña Auditoría verifica consistencia estructural; Indicadores resume estado, recorridos y métricas.

El archivo masivo identifica subárboles completamente elegibles: todos sus eventos deben ser de prioridad baja y tener antigüedad estrictamente mayor que T horas. Se elige el mayor subárbol; los empates se resuelven por mayor profundidad de raíz y luego mayor identificador. Los eventos archivados conservan su identidad y pueden reactivarse mediante una revisión nueva.

8. Interfaz

Pestaña	Uso principal
Árbol AVL	Explorar el árbol, niveles, nodos, balance y rotaciones.
BST	Comparar la estructura balanceada con la no balanceada.
Plano	Ver eventos, zonas, referencias y selección espacial. El encuadre se ajusta a los eventos.
Eventos	Consultar el catálogo activo y seleccionar eventos.
Cola	Preparar, importar y procesar reportes FIFO.
Histórico	Revisar archivados/eliminados, previsualizar archivo y consultar el registro deshacible.
Archivos	Guardar/cargar estados, cargar inserciones y administrar versiones.
Auditoría	Ejecutar verificaciones e inspeccionar métricas y recorridos.
Consultas	Ejecutar búsquedas del dominio.
Indicadores	Consultar contadores y estado agregado del catálogo.

El panel izquierdo contiene las operaciones rápidas y puede desplazarse verticalmente. El panel derecho muestra el detalle del evento seleccionado y la tabla de activos.

9. Persistencia e importación

La pestaña Archivos ofrece guardar estado completo, cargar por topología, cargar por inserciones y administrar versiones con nombre.

Tipo JSON	Uso
sismolab-state	Estado completo, topología del árbol, catálogo, cola, zonas y parámetros.
sismolab-insertions	Eventos a insertar; el árbol se construye a partir del orden de inserción.
sismolab-reports	Ráfaga de reportes para agregar a la cola.
sismolab-version	Snapshot nombrado para restaurar una versión.

La carga valida tipo, estructura, valores, árbol, zonas, prioridades y referencias antes de restaurar. Un archivo rechazado conserva el escenario anterior y muestra los problemas detectados. La escritura JSON se realiza mediante archivo temporal y reemplazo para evitar dejar archivos parciales.

Las cargas de topología reemplazan el escenario completo; no agregan zonas o eventos a la sesión existente. Guarde el estado actual antes de cargar otro escenario si desea conservarlo.

10. Pruebas y archivos de ejemplo

Desde la raíz del repositorio:

```
python -m unittest discover -v
python -m scripts.run_cases
```

El primero ejecuta la suite de pruebas. El segundo ejecuta casos reproducibles de límites, reportes, rotaciones, archivo y persistencia e informa esperado frente a obtenido.

Para regenerar los JSON de data/:

```
python -m scripts.build_data
```

La generación es determinista, pero vuelve a escribir los archivos de ejemplo; no la ejecute si ha modificado esos datos y quiere conservar los cambios.

Archivos destacados:

- data/topologia_normal.json: topología AVL normal, estaciones y zonas de ejemplo.
- data/topologia_estres.json: escenario de estrés.
- data/topologia_inconsistente.json: archivo diseñado para comprobar el rechazo de una topología inválida.
- data/estado_inicial_rafaga.json: estado base de la ráfaga de demostración.
- data/rafaga_mixta.json: reportes de alta, confirmación, corrección, reporte antiguo, conflicto y datos inválidos.
- data/carga_inserciones_*.json: órdenes aleatoria, ascendente y descendente.

Para la ráfaga mixta, cargue primero estado_inicial_rafaga.json por topología y luego rafaga_mixta.json desde la pestaña Cola.

11. Referencia rápida de módulos

Módulo	Responsabilidad
core/avl.py	Inserción/eliminación balanceada, estadísticas de rotación y recuperación.
core/bst.py, core/binary_tree.py	BST y operaciones compartidas de árbol.
core/fifo.py, core/stack.py	Cola FIFO enlazada y pila enlazada.
domain/catalog.py	Catálogo, validación de eventos, prioridades y asociaciones.
domain/reports.py	Decisión de negocio para un reporte.
domain/archive.py	Planificación y ejecución de archivo masivo.
domain/queries.py	Consultas medidas sobre el catálogo.
services/scenario.py	Transacciones, undo, cola y coordinación del escenario.
services/snapshot.py	Captura y restauración de estado/topología.
services/persistence.py	JSON: lectura, validación, importación y exportación.
ui/controller.py	Frontera entre UI y servicios; resultados y mensajes.
ui/app.py	Construcción de pantallas y coordinación de acciones visuales.
tests/	Pruebas de estructuras, reglas, persistencia, controlador e interfaz.